

DATABASES · RETRIEVAL CORRECTNESS · DISTRIBUTED CONSENSUS

One field, five operations

Part II — when the index is current: freshness as a correctness property, and what a change of consensus protocol does not touch

David Leconte — Customer Success Engineer, IBM France | Software, Data & AI | Horizon
Part II of a diptych. Roughly twenty minutes to read; the measurement at the end takes fifteen.

ABSTRACT — PART II OF TWO

Where index maintenance sits relative to the commit decides whether a system can read back what it just wrote. One engine pays for indexing on the write path and is searchable at acknowledgement; the other maintains its search indexes outside the transaction commit window, asynchronously, and is faster to write for it. Neither is a defect: it is the same cost placed in two different places.

Two kinds of index have to be separated before that claim means anything, and a comparative campaign forced the separation. Ordinary secondary indexes are maintained synchronously by both engines, so the index is current on both and on that question they are at parity — though not on what a caller waits for, which ranges from 7 ms to 894 ms across the three configurations measured. The asymmetry is confined to the *search and vector* index, and there it has been measured: a document just written is returned by search on the first query on one engine, and not for two thirds of a second on average on the other — a periodic refresh spread from a tenth of a second to a second and a fifth — while remaining findable by its identifier throughout. Whether that

costs anything depends on how soon the search arrives: sweeping that delay shows the asynchronous path missing every document at zero, half of them at half a second, and none at all beyond one. This is the only axis of eight in this diptych that the decomposing engine wins, and the only one it wins on an exactly tested result — a miss-rate, the latency ranges overlapping and establishing nothing: absolutely inside the comparator's refresh window and not at all outside — a window that measured about a second on this deployment and is a property of that deployment, not of the product. Narrower, and more conditional, than earlier revisions of this part claimed.

For a catalogue the difference is invisible. For a system that composes a write and a search without waiting, it is categorical. A system that records a fact and cannot retrieve it is producing a wrong answer, not a slow one, and the failure is invisible to latency instrumentation by construction. This part argues that the write-to-searchable interval belongs in acceptance criteria rather than in a performance backlog, and supplies a control that lets a reader put their own number to it.

It then qualifies its own side's guarantee. The Data API's consistency levels are configuration rather than fixed behaviour, and they are asymmetric: writes commit at quorum while vector search reads a single replica. Measured on a co-located ring the resulting window was not detectable; a wide-area deployment is where it would surface, and this bench cannot reproduce that.

The part closes on the fifth operation — the consensus round guarding a conditional write, now priced at three replicas — and on its designated successor in Apache Cassandra, which turns out to leave the four storage steps of **Part I** untouched.

A note on epistemic status. Comparative writing about competing databases fails most often not through malice but through the silent promotion of plausible mechanism to documented fact. Every substantive claim below therefore carries

one of four markers, and the appendix records the exact passage on which each rests.

D Documented. Established by the vendor's own version-pinned documentation, by the project's source of record, or by peer-reviewed publication. Quoted verbatim in the appendix.

R Reported. Described publicly by the engineers who built the system, or by the upstream project, but not restated in the versioned documentation of the shipped product. Directionally reliable; not a contractual guarantee.

U Unverified. Asserted here on structural grounds, or widely repeated in the practitioner literature, but not established by any source of acceptable rank that I was able to consult. Treat as a hypothesis to be tested against your own build.

M Measured. Observed on a named build under stated conditions, by a harness reproduced in the appendix. Stronger than **U**, weaker than **D**: it establishes what one build did on one day, not what the vendor commits to. Every **M** claim names its measurement (M1–M4) and every measurement names its conditions.

Nota bene on the figures. A diagram carries more apparent authority than a paragraph: it resembles an architecture specification even when it is an argument. One of the seven figures below reports a measurement taken on a named build; it is tinted green and labelled as such. The other six are schematics — they organise an argument and measure nothing, whatever their rectangles may suggest. Every caption additionally states which of its elements are documented, reported or measured. Where a figure depicts something marked **U**, the caption says so, and the box should be read as a hypothesis drawn in rectangles rather than as a contract.

1. What decomposition purchases

Three properties, of which the third is the one comparative treatments most consistently omit.

Queryability without index design. Already treated, and genuinely valuable early in a product's life, when the schema is still an argument rather than a contract.

Write scale-out without a coordinating primary. No single node must accept every write, so capacity is added by adding nodes rather than by enlarging one.

An index current at the moment the write completes. D Because indexing occurs on the write path, a document becomes searchable as soon as it is written. The HCD 2.0 documentation states this directly, and attributes it to the vector index implementation by name:

"When you insert data, JVector adds those new documents to the graph immediately, so you can efficiently search right away."

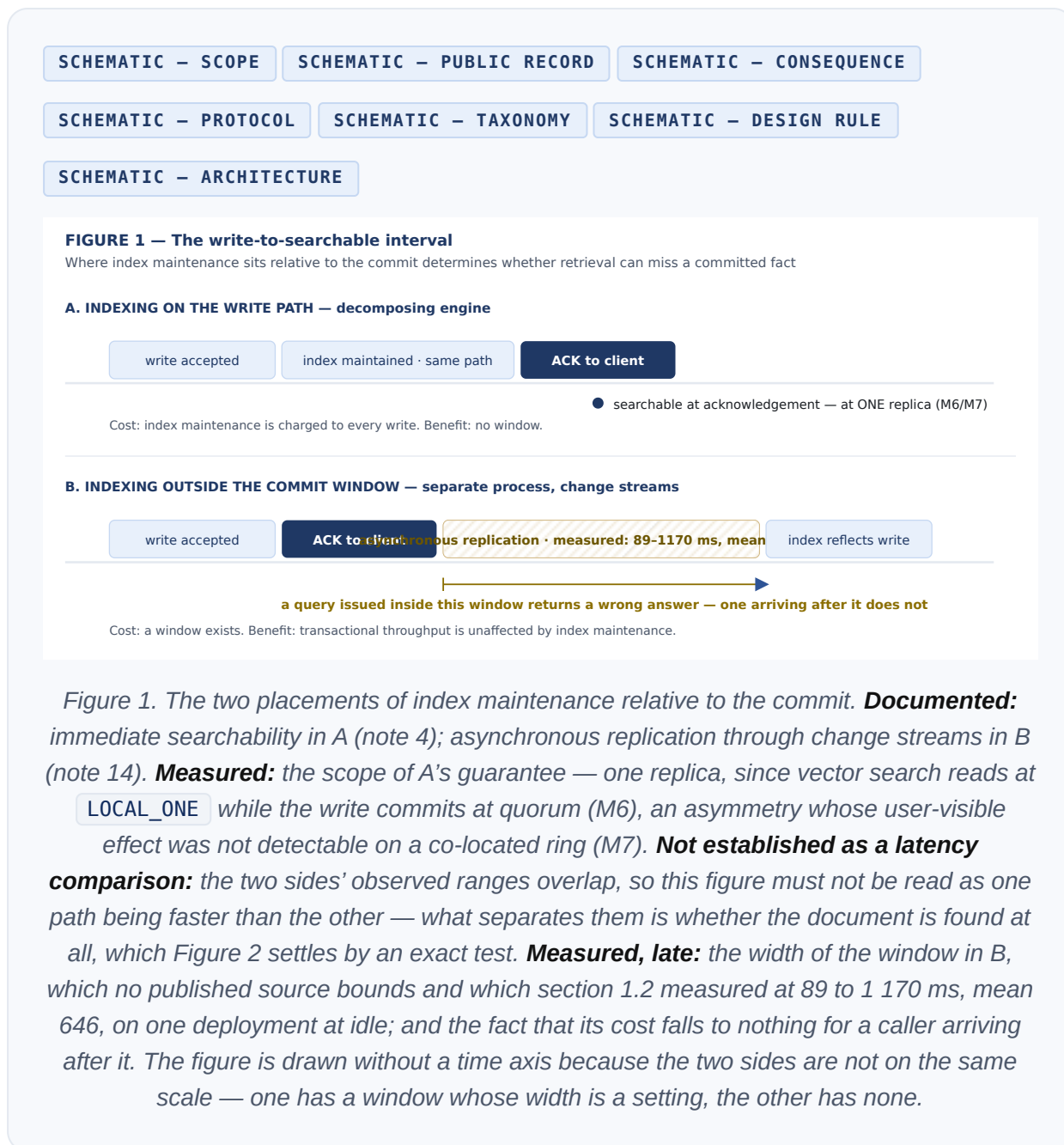
DataStax, *About vector databases*, Hyper-Converged Database 2.0.⁴

M One qualifier that sentence omits, and it only appears above a single replica. The Data API's consistency levels are configuration properties rather than fixed behaviour, and its defaults are asymmetric: writes and ordinary reads commit at LOCAL_QUORUM, but *vector search reads at* LOCAL_ONE.³⁹ "Searchable as soon as it is written" is therefore a single-replica guarantee, not a quorum one: at $RF > 1$ a vector query issued at acknowledgement can in principle read the one replica the write has not yet reached.

M Measured on a six-node, two-datacentre ring with the collection at three replicas, the effect was **not detectable**: in 120 of 120 cycles the just-written vector was the top match on the first LOCAL_ONE query, and in no cycle did the quorum key read succeed while the vector search missed.⁴⁰ Sub-millisecond inter-replica latency closes the gap far below the round trip that carries the query. The window is therefore real in the protocol and bounded below the measurement floor on that ring — and a wide-area deployment, where inter-replica latency is tens of milliseconds, is precisely where it would surface. Which is the same window this article attributes to the other engine, arriving here by a different route and at a different scale. The guarantee is real; its scope is one replica; and a reader deploying across datacentres should measure it there rather than inherit it from this page.

1.1 Index freshness as a correctness property

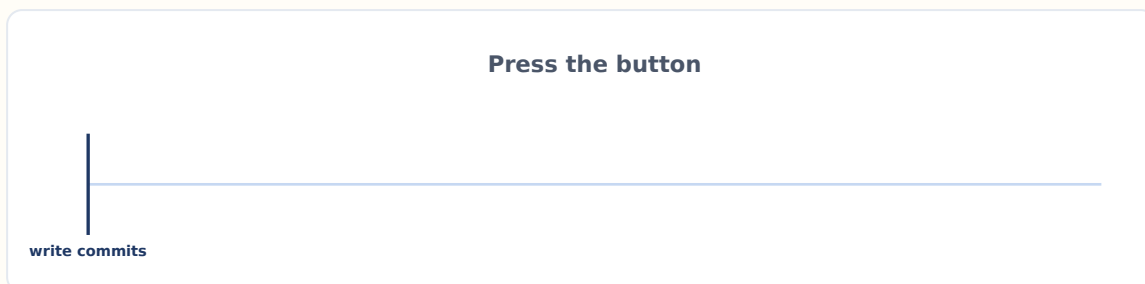
Depending on what is being built, this third point changes character entirely, and it changes categorically rather than by degree. Since the whole of it follows from a single structural choice, namely where index maintenance sits relative to the commit, it is worth seeing before it is argued.



A window either catches your read or it does not, and which of the two happens is decided by two numbers that no diagram can supply on your behalf. The control below takes them from you.

DEMONSTRATOR — YOU SUPPLY THE NUMBER**Where does your read land? — interactive; static values shown**

The mechanism is what is being shown; the magnitude is yours. One thing the control makes visible that prose does not: move the read past the lag and the failure disappears entirely, which is exactly what section 1.2 measures — the window costs nothing to a caller who arrives after it.



Nothing here is a measurement. On the build this article did instrument, no index lag was detectable below a floor of roughly one network round trip; for any system you have not instrumented, the honest value is unknown, which is the reason this control exists.

For a product catalogue, several seconds of index lag is invisible; no user experiences it and no requirement references it. For anything that reads back what it has just written, it is not a performance detail at all: a system that records a fact and cannot retrieve it is producing a wrong answer, not a slow one, however promptly the index subsequently converges. How soon the read arrives decides whether that matters, and section 1.2 puts a number on it. The failure is semantic, it is invisible to latency instrumentation, and it will be reported by users as hallucination rather than as staleness. *A fortiori*, any system that writes and reads within a single conversational turn inherits the interval as a hard correctness bound rather than as a service objective.

M Measured, that interval turns out not to be an index interval at all. Across forty idle cycles on HCD 2.0.6, the freshly written marker was found on the *first* attempt forty times out of forty; the attempt histogram has a single entry.³⁸ Decomposed, the 22.9 ms median splits into an insert of 9.8 ms and the one retrieval of 11.2 ms that followed it: two HTTP round trips, and nothing else. No index lag was detectable at all, which means the harness's own threshold — it declared the probe inconclusive for exceeding 50 ms under load — was measuring transport rather than indexing. A harness that reports the wrong thing precisely is still reporting the wrong thing, and the attempt histogram is what caught it.

Two caveats bound that result; none softens it. The retrieval exercised a storage-attached index on a scalar field, not the vector graph the quotation above concerns; and one node at RF = 1 removes any replication interval by construction. The later vector probe, at three replicas, used vectors of sixteen dimensions — between one and two orders of magnitude below the embeddings a retrieval system

actually stores — and the cost of maintaining a graph index synchronously at production dimensions was not measured. What is established is that on this build, on this path, the write-to-searchable interval is indistinguishable from the cost of asking.

If you are building a system of that kind, the write-to-searchable interval belongs in your **acceptance criteria**, not in your performance backlog.

M A distinction has to be drawn here that earlier revisions of this part did not draw, and drawing it removes most of the territory this section thought it held. There are two kinds of index in play, and only one of them is contested. *Ordinary secondary indexes* are maintained synchronously by both engines: a later comparison wrote a document and queried it through one, on a million-document collection, and all three arms found it on the first attempt, forty times out of forty.⁴¹ The index was current everywhere, and on *that* question there is parity.

M The elapsed time is not at parity in either direction, and an earlier revision of this section lost the distinction by calling the whole thing a tie. Writing a document and then retrieving it through a secondary index took 894 ms against MongoDB as ordinarily deployed, 33 ms against the decomposing engine, and 7 ms against MongoDB under a wildcard index.⁴¹ Twenty-seven times faster than the first, four and a half times slower than the second.

Which is the distinction this part exists to draw, arriving from an unexpected direction. *Index currency* — whether the write is in the index yet — is a tie on ordinary indexes. *Time to answer* is not, and for a user or an assistant waiting on a reply, the second is what the first feels like. Against the configuration a team actually deploys, the automatic index turns almost a second into a thirtieth of one. Against a configuration that declared its index, it does the opposite.

The asymmetry this part describes concerns the other kind: the *search and vector* index, synchronous on one engine and maintained asynchronously through change streams on the other. That is the axis where the window exists, and it is where the argument of this section lives.

M That axis has now been measured, on one host in one session, and it is the only axis in this diptych where the decomposing engine wins decisively. MongoDB's search engine turns out to be deployable locally after all, through the vendor's own packaged image, which runs the database and the search process together — and which the vendor designates for evaluation only, a single node without high availability, not a production target.^{42 48} That designation is the vendor's own, and it applies to every freshness figure in this section. **M** The engine inside that image has since been pinned: `mongot` 1.75.1, above the 1.70.1 at which the self-managed capability reached general availability, so the figures below were not taken against a preview build. **D** They were taken against something the image labels itself: edition `localDev`, alongside a Community-edition `mongod` 8.3.11, at image digest `sha256:e118f5c131c5004e4ccd0554d3f32b048b4af6a33325e80e424eaa1c8986d8f1` built on 16 September 2026.⁵⁰ That label narrows the reserve rather than lifting it. The refresh interval measured

here is the default of a *development edition* of the engine, and whether the editions a reader would deploy — the managed service, or a self-managed Community or Enterprise process — commit on the same period is not established by anything in this study. ^D The topology differs too: on the managed service the vendor offers Search Nodes, dedicated infrastructure that runs the search process apart from the database and is generally available on the three major clouds,⁵¹ whereas what was measured here is that process as a sidecar sharing one container with the database. A dedicated node may refresh faster, or less often; the study cannot say which. With it running and a search index reported ready, the write-to-searchable interval is no longer an argument.

Path	Write to searchable (median)	Found on the first query?
Decomposing engine, search index on the write path	45 ms	yes — median one attempt
MongoDB <code>\$search</code> , index maintained asynchronously	646 ms mean , spread 89–1 170	no — the index is not yet current
MongoDB fetch by identifier, for contrast	1.3 ms	yes

^M That second row took two attempts to measure, and the first one was mine to get wrong. A probe that writes, then polls until the document appears, then writes again, synchronises itself to the indexing cycle: every write lands just after a commit and waits a full period. It returned a tight 1 015 ms, and an earlier revision of this section published it. Inserting a random delay before each write, so that the cycle is sampled at every phase, gives a different picture — a lag spread from 89 ms to 1 170, mean 646, standard deviation 291.⁴³ The signature is a periodic refresh of roughly a second rather than a fixed pipeline delay, and my first figure overstated it by about 1.6 times.

A document that has just been written is therefore *not findable by search* on MongoDB for something between a tenth of a second and a second and a bit, depending on where the write falls in that cycle, while being immediately findable by its identifier — which is the shape of the failure this section opened with, measured rather than posited. The decomposing engine shows no such window, and the precise form of that statement matters: forty cycles out of forty returned the document on the first query, which bounds its index lag *below the measurement floor* of one HTTP round trip.⁴² The lag may be nothing; it may be forty milliseconds. This measurement cannot tell the two apart, and an earlier revision of this section asserted the first — a temptation the same volume of evidence had already taught the rest of this diptych to resist.

^U Read as a comparison of latencies, that table does not establish a direction at all. The synchronous side's slowest observation, above half a second, exceeds the asynchronous side's fastest, at 89 ms; the supports overlap, and the interval of the ratio runs from 0.16 to 36.7 and contains one.⁵⁴ The only version of this comparison whose ranges separate is the self-synchronised one this

section withdrew. What establishes the axis is not the latency but the proportion in section 1.2 — thirty misses in thirty against none, exactly tested — and a reader should carry that distinction: the freshness advantage is a fact about whether a search finds the document, not about how long either search takes.

U The winning side is also the more erratic one, which a single median conceals. Against a median of 45 ms the decomposing engine shows a ninety-fifth percentile of 85 and a maximum above half a second, while the asynchronous path, slow throughout, varies by a standard deviation of only 35 ms around its thousand. One engine is quick and uneven; the other is late and dependable. For a system whose failure mode is a wrong answer rather than a slow one that trade still favours the first, but it is a trade and not a clean win.

A write followed immediately by a search finds the document on one engine and misses it, on average, for two thirds of a second on the other — sometimes for a tenth, sometimes for a second and a fifth, and it cannot know which.

1.2 How much of that matters, and to whom

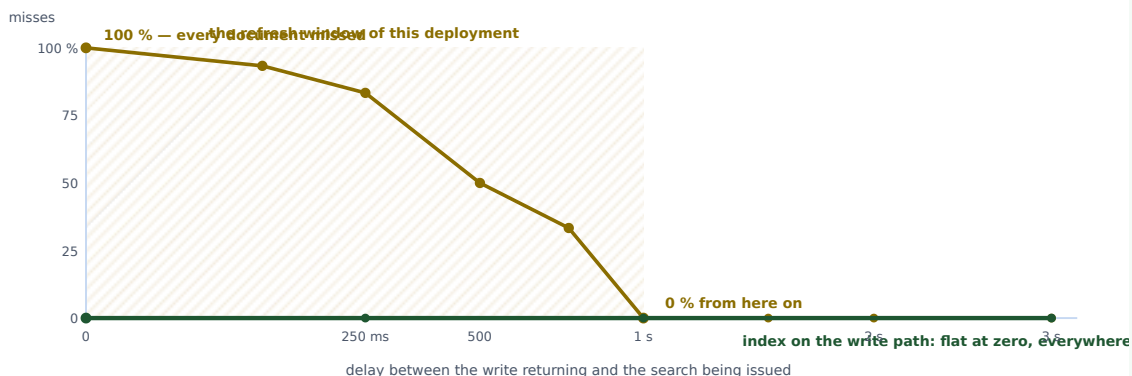
M A lag only costs you something if you read inside it, so the question that decides this section is not how long the window is but how soon the read arrives. Sweeping the delay between write and search, thirty cycles at each point, gives the failure rate directly.⁴⁴

Search issued after	MongoDB <code>\$search</code> misses	Decomposing engine misses
immediately ⁴⁴	100 %	0 %
100 ms	93 %	0 %
250 ms	83 %	0 %
500 ms	50 %	0 %
750 ms	33 %	0 %
1 000 ms and beyond	0 %	0 %

MEASURED ON ONE BUILD

FIGURE 2 — What the window costs, against how soon you read

Share of just-written documents a single search fails to return. Thirty cycles per point. Delay axis is square-root scaled to keep the sub-second det.



Inside the window the asymmetry is total; outside it there is none. The window is a setting on one side and does not exist on the other — which is the whole of the fi

Figure 2. **Measured, not documented** (appendix, measurement M19). Thirty cycles at each of nine delays, both engines on one host. **Read with three qualifications:** the zero-delay point compares a native driver against an HTTP interface and so not the same instant, worth about three percentage points; the transition is located only to within a quarter of a second, the flanking points being 750 ms and 1 000; and thirty cycles gives roughly eighteen points of confidence interval around the middle values, so the shape is solid and the intermediate readings are not. The shaded region is this deployment's refresh window; another's would be wider or narrower, and the curve would move with it.

M Read the first row before the caveats, because it is the finding. A search issued the instant a write returns misses every document on the asynchronous path and none at all on the synchronous one. At half a second it still misses one in two. From a second onward it misses nothing — and that last transition is where the argument of this part turns.

D One asymmetry is not a matter of deployment and should not be phrased as if it were. The window on the asynchronous side has a width, and that width is a setting — this deployment's happens to be about a second, another's would differ. The synchronous side has no window to widen or narrow: its index is maintained on the write path, so there is no interval, no setting that would create one, and nothing to tune. What the two deployments differ in is how much the asymmetry costs. That it exists is architectural.

U Three qualifications on the table itself, before its consequences. The first row does not compare the same instant: the comparator was queried through a native driver, which reaches its index about a millisecond after the write, while the decomposing engine was queried over HTTP, which reaches its own some forty-five milliseconds later. Asking one earlier than the other flatters the later, and correcting for it would read about 97 % rather than 100 — a small effect, but it is the same driver-against-interface confound that distorts the read axis in Part I. Second, the transition is located only to within a quarter of

a second: the flanking points are 750 ms at a third and 1 000 ms at nothing, with nothing measured between. Third, thirty cycles per point puts a confidence interval of roughly eighteen points around the middle values, so the shape of the curve is solid and its intermediate readings are not.

U Which obliges this part to withdraw the example it opened with. The assistant of the opening paragraph — write a fact, answer the user, search for the fact on the next turn — does not experience this failure, because a turn driven by a language model takes seconds and the window closes inside one — **U** where “takes seconds” is an assumption about such turns that this study states and did not measure against any real workload. Every earlier revision of this section used conversational retrieval to motivate the argument. On these numbers that motivation is wrong: by the time the model has finished speaking, the refresh has happened and the document is there.

M What the measurement does establish is narrower and sharper. Inside the window the asymmetry is not a tendency but a certainty: at the shortest delay the asynchronous path misses *every* document and the synchronous one misses none, and the failure rate is still one in two at half a second. Whether a given workload sits inside that window is the only question that matters, and it is a question about the workload. One term of it deserves naming. In a retrieval system the document is written with its vector already computed, so embedding the document sits *before* the write and does not enter the window; embedding the *query* sits inside it and consumes part of the interval — **U** plausibly tens to a few hundred milliseconds, an estimate this study did not measure — without closing it. A pipeline that writes the raw document and computes its vector asynchronously moves the start of the window to the vector’s arrival rather than the document’s. **D** The comparator now offers exactly such a pipeline as a feature: with its Automated Embedding, in preview for self-managed deployments, the search process itself generates the vector at index time rather than the application supplying it.⁵² A document written that way enters the search index only after both the embedding and the refresh, so the convenience widens the window this section measures rather than narrowing it — a point in the decomposing engine’s favour that the measurements here, taken with vectors supplied by the client, do not capture.

U The boundary is not a second. It is *this deployment’s refresh interval*, which happens to be about a second and which no user setting exposes.⁴³ On a deployment refreshing every two hundred milliseconds the advantage would be total below two hundred milliseconds; on one refreshing every three seconds it would reach far further. The curve traces the interval, it does not fix it, and stating the finding as “below a second” would make a container default sound like a property of a product.

One engine has a window whose width is a setting. The other has none, and no setting would give it one. What your deployment decides is how much that costs you — not whether it is true.

U The margin, stated at its narrowest rather than its widest: even in the best phase of the cycle, the asynchronous path takes 89 ms to become searchable, which is already above the 45 ms floor below which the synchronous one cannot be resolved. The mean gap is about fourteen times; the best-case

gap is about two. Both favour the synchronous index, and the smaller number is the one a design should be built on.

U Four reservations, and the first two matter. The packaged image runs a single-member replica set, so a majority write concern is satisfied trivially there. An earlier revision then argued that the asynchronous lag is a property of the ingestion path rather than of the replica count, and is therefore representative. That reasoning inverts itself: a quantity that depends on the deployment is precisely one this deployment cannot vouch for elsewhere. The image is, besides, the vendor's development and testing build, whose ingestion tuning has no particular reason to match the managed service a reader would actually buy. What is established is that the path is asynchronous and that the lag on this build, at idle, is about a second. Whether a production deployment lags by a hundred milliseconds or by five is not established here. The two systems were also exercised on different search paths: a vector index on one side, a text index on the other. That the two ingestion architectures are the ones being compared is an inference, and a reasonable one, but it is not what was measured. The strict pairing would set the vector paths against each other, or the two text paths; both exist on both engines, and neither pairing was run. The thousand milliseconds is an idle figure and load would widen it. And both engines were cache-resident, a condition Part I shows to be decisive elsewhere.

1.3 The opposite resolution, stated in the vendor's own terms

D MongoDB's self-managed search and vector search execute in a separate process, `mongot`, running alongside the database.¹⁵¹⁶ MongoDB describes the mechanism precisely:

"mongot builds and maintains these compute-intensive indexes outside of the transaction commit window by leveraging change streams to asynchronously replicate changes to source data, allowing customers to add search and vector search capabilities without fear of impacting existing transactional workloads."

MongoDB, *Now Source Available: The Engine Powering MongoDB Search*, 15 January 2026.¹⁴

This is a deliberate and defensible design, and it is panel B of the figure above. It removes index maintenance from the write path, which is precisely the cost the decomposing engine elects to pay. Two coherent answers to one question, with the trade legible in a single sentence from each vendor, which is a considerably better basis for a decision than any benchmark.

Note that the two designs are not in competition on a single axis. They optimise different things and the choice is properly determined by the workload, not by preference: *ceteris paribus*, a catalogue tolerates the window and profits from the throughput; a retrieval system does not tolerate the window at all.

D One corollary follows from the quoted mechanism and requires no further source: asynchronous replication of changes implies a non-zero interval between a write committing and the corresponding search index reflecting it. **U** What that interval is, and whether MongoDB documents any bound on it, I did not establish. This is again a measurement question rather than a reading question.

D Status matters here and is easy to misstate, because three things carry the same name. The *managed* service already offered these capabilities as production-ready when the self-managed preview opened in September 2025.⁴⁵ The *self-managed* capability — the same engine deployed alongside one’s own database — went from public preview in October 2025 to general availability at `mongot` 1.70.1,⁴⁶ and **R** a vendor FAQ marks the 0.x builds as preview releases not recommended for production.⁴⁹ And the engine’s *source* was released under the Server Side Public License as a public preview on 15 January 2026, which is a statement about the code’s availability rather than the feature’s.¹⁴ An earlier revision of this paragraph conflated the second and third and could not confirm the first; the vendor’s pages resolve all three. What the vendor also states, and this article relies on: the search engine is the same on the managed service and on a self-managed deployment, so query behaviour, scoring and index semantics are, in the vendor’s word, *generally* consistent across them.⁴⁷

2. Where JVector actually appears — three loci, three statuses

A persistent conflation in this area will invalidate an evaluation if carried into it. The same library appears in three distinct places with three distinct statuses, and one place where it does not appear at all.

D In Apache Cassandra 5.0 upstream: it does not. Upstream vector search through storage-attached indexing was specified and built on Lucene's Hierarchical Navigable Small World implementation. The enhancement proposal is explicit: the implementation leverages Lucene's HNSW library, characterised there as the best available ANN algorithm for Java at the time.²¹ Storage-attached indexing itself is a 5.0 feature.²²

D In the DataStax distribution: as the shipped engine. JVector, a graph index of the DiskANN family supporting product and binary quantisation,¹⁹ is part of what the distribution adds atop the Cassandra 5.0 base,⁵ and the HCD documentation names it as the mechanism behind immediate searchability.⁴

D In OpenSearch: as an official plugin, not a core engine. The documented core k-NN engines are Lucene, Faiss and the deprecated NMSLIB, the latter superseded in favour of the first two;¹⁷ the default engine changed from NMSLIB to Faiss in the 2.18 line, and index creation on NMSLIB is blocked from OpenSearch 3.0.¹⁸ JVector is available separately as `opensearch-project/opensearch-jvector`, Apache-2.0 licensed and governed within the OpenSearch project, with a release train tracking OpenSearch versions.²⁰ Its README claims thread-safe concurrent index construction, incremental merges in place of full graph rebuilds, and — directly relevant here — compatibility with Cassandra, "which allows to more easily transfer vector encoded data from Cassandra to OpenSearch and vice versa."¹⁹

The practical consequence: a benchmark of "Cassandra vector search" tells you nothing about the DataStax distribution, and a benchmark of either tells you nothing about the OpenSearch plugin. Three artefacts, three governance models, three release cadences. Establish which one your evaluation actually exercised before you read its numbers.

FIGURE 3 — Three loci of the same library, and one place it does not appear

Conflating these three invalidates an evaluation before the first measurement is taken

APACHE CASSANDRA 5.0	DATASTAX DISTRIBUTION	OPENSEARCH 3.x
upstream · Apache Software Foundation	HCD 2.x · based on Cassandra 5.0	separate project · separate governance
Vector ANN Lucene HNSW	Vector ANN JVector — shipped engine	Core k-NN engines Lucene · Faiss · NMSLIB
Index framework SAI — new in 5.0	Family · compression DiskANN · PQ and BQ	NMSLIB deprecated; default moved to Faiss at 2.18
JVector: NOT PRESENT specified on Lucene HNSW — CEP-30	Added by the distribution atop the 5.0 base — immediate indexing	Separate plugin, not a core engine opensearch-project/opensearch-jvector Apache-2.0 · release train tracks OpenSearch

Figure 3. The three loci. Every element shown is **documented**: Lucene HNSW upstream (note 21), SAI as a 5.0 feature (note 22), the distribution's base version (note 5), the DiskANN family and quantisation support (notes 19 and 4), the OpenSearch core engines and the NMSLIB deprecation (notes 17 and 18), and the plugin's licensing and release train (notes 19 and 20).

3. The fifth operation and its designated successor

Of the five steps above, the last is a consensus round, and it alone has a named successor already in flight. Any reader holding a current view of the Cassandra roadmap will ask about it. The answer is more interesting than a simple replacement narrative, and it is worth developing with some care, because it both qualifies and reinforces the thesis of this article.

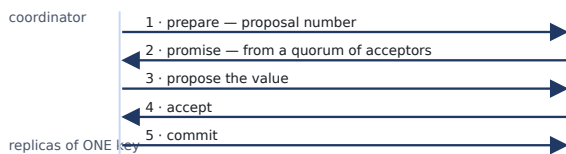
3.1 What Accord specifies

D CEP-15 proposes Accord, a leaderless timestamp protocol for general-purpose distributed transactions. Its stated goals are transactions operating over any set of keys atomically, strict-serializable isolation, one wide-area round trip under normal conditions, failure tolerance unaffected by any minority of replica failures, absence of any introduced bottleneck, and — the goal most relevant here — support for live migration from Paxos. Two techniques are proposed as novel: a *reorder buffer* guaranteeing single-round-trip consensus under a timestamp protocol, and *fast path electorates* preserving leaderless fast-path availability under worst tolerated failure. Timestamps are hybrid logical clocks made globally unique by appending each replica's identifier. The proposal situates itself in the EPaxos lineage, claiming better properties than Caesar or Tempo.²³

The single round trip is a *design goal inscribed in the proposal*, not a measurement. **U** No benchmark figures published by the Apache Software Foundation were located. Practitioners will encounter a great many performance claims about Accord in secondary material; none that I could trace to a primary source.

FIGURE 4 — Two ways to arbitrate a write

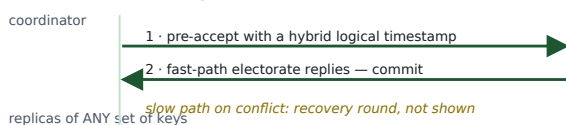
Phases as documented. No latency is shown here: none is published for either protocol, and the one this study measured for Paxos at three replica

A. PAXOS LIGHTWEIGHT TRANSACTION — in the field today**Scope: one partition key.**

Multi-key atomicity is the application's problem.

measured: `LOCAL_SERIAL` · 3 client retries

A leader is elected per proposal; contention on a hot key serialises through it.

B. ACCORD — CEP-15, alpha**Scope: any set of keys, atomically.**

Leaderless — no elected coordinator to serialise on.

Strict-serializable isolation.

One wide-area round trip is a STATED GOAL of the proposal, not a published measurement.

Panel A phases from the vendor's Paxos documentation; consistency level and retry count measured on one build. Panel B from CEP-15 only. Neither panel as

Figure 4. Arbitration, not storage. **Documented:** the Paxos phase structure (note 13); Accord's leaderless character, strict-serializable target, hybrid logical timestamps, fast-path electorate and multi-key scope (note 23). **Measured:** `LOCAL_SERIAL` and the three-retry default on one build (notes 38 and 1). **Explicitly not shown:** any latency figure for either protocol. The single round trip in panel B is a design goal inscribed in the proposal; the Apache Software Foundation has published no measurement of it, and practitioner claims to the contrary trace to no primary source.

3.2 Coexistence, not replacement

D This is the point most frequently mangled in summary. The proposal is explicit on both horizons:

"This work is expected to replace Paxos for the project. In order to support live migration both protocols will be maintained for at least an interim period, after which it is expected that Paxos will be deprecated. The precise timing of this will be up to the community."

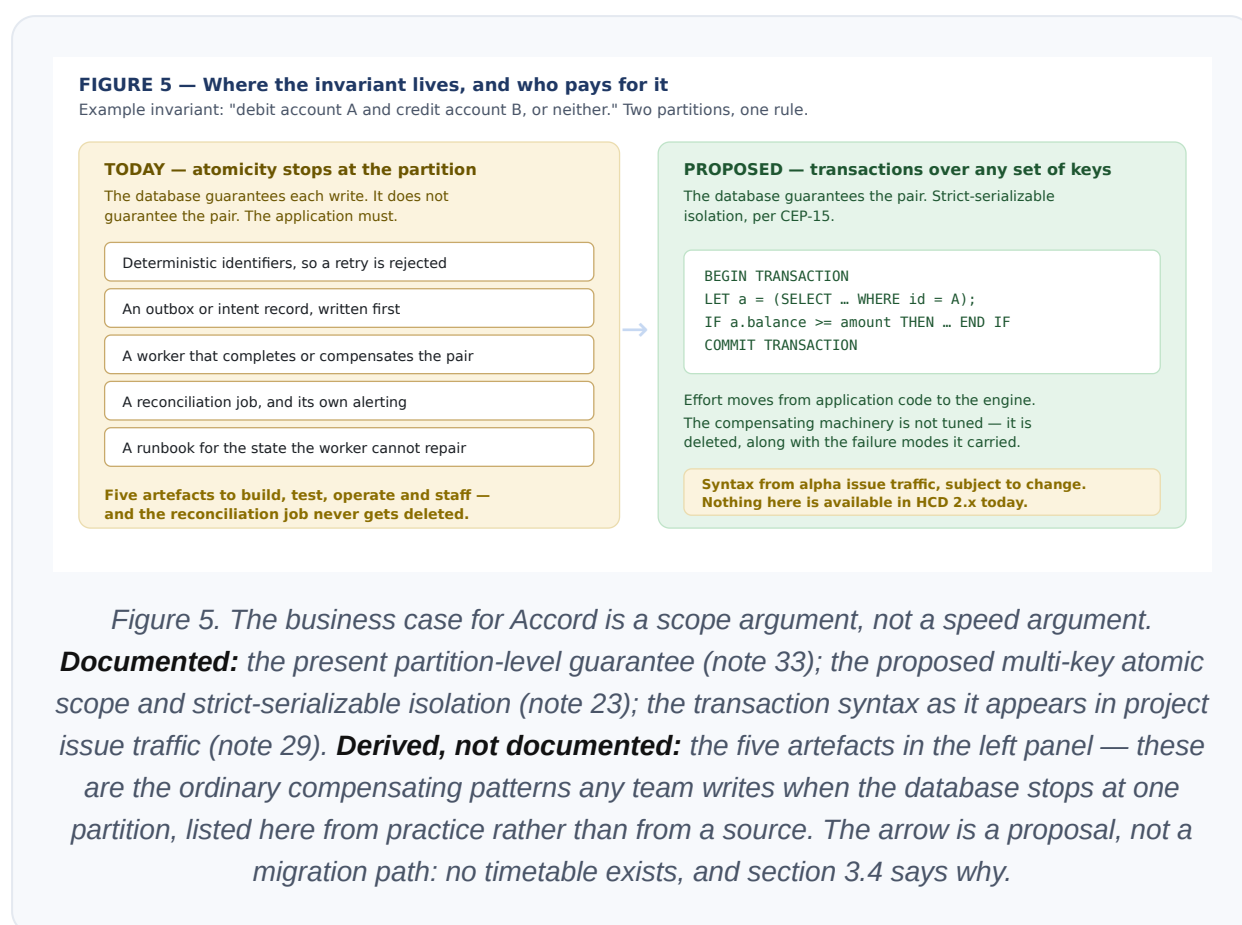
CEP-15, *General Purpose Transactions*, section "Long Term".²³

And, on the architecture of the coexistence: supporting the Paxos-based lightweight transaction and Accord side by side is cited in the proposal as an instance of the intended modularity.²³ No deprecation of Paxos has been announced. Lightweight transactions remain the mechanism in the field.

3.3 What multi-key atomicity would change for an application

The architectural difference in Figure 4 has an application-level consequence that is worth drawing explicitly, because it is where the business value of Accord actually sits — not in speed, about which nothing is published, but in *what the database is willing to guarantee*.

D Today the guarantee stops at the partition: HCD does not adhere to ACID transactions with rollback or locking mechanisms, and a write is atomic at partition level.³³ Any invariant spanning two records is therefore held by the application, in code someone must write, test and operate. **D** CEP-15 proposes transactions over any set of keys, atomically, at strict-serializable isolation.²³ The following figure maps that shift onto the compensating machinery it would remove.



3.4 Release status, and the four things nobody can tell you

D Apache Cassandra 6.0-alpha1 is tagged 3 April 2026 and 6.0-alpha2 is tagged 7 August 2026 in the project repository.²⁴ The release announcement characterises the build without ambiguity: a quarterly alpha on the 6.0 series, and quarterly alphas are intended only for development and testing purposes.²⁵

D The `NEWS.txt` of the 6.0-alpha2 tag enumerates, under the 6.0 new-features heading: JDK 21 with generational ZGC as default collector, CEP-37 Auto Repair, CEP-21 Transactional Cluster Metadata, CEP-42 Constraints Framework, CEP-24, CEP-43 and CEP-55, unified-compaction parallelisation, JMX configuration in YAML, and several new CQL functions. **Accord is not among them.** This absence is suggestive but not probative: the same file carries an explicit editorial placeholder, and its upgrade section still refers to the release as 5.1, evidencing a renumbering not yet reflected throughout.²⁶

D The tracking epic for CEP-15 is titled "Accord Beta" and stands, at consultation, In Progress and Unresolved; the linked ticket for writing the CEP-15 documentation is likewise In Progress.²⁷ **R** Documentation under review in the project describes the activation model: a global switch in `cassandra.yaml` (`accord: enabled: true`) and a per-table property (`WITH transactional_mode = 'full'`), under which ordinary CQL operations on that table execute through Accord.²⁸ The explicit transaction surface — `BEGIN TRANSACTION`, `LET`, conditional block, `COMMIT TRANSACTION` — appears in project issue traffic.²⁹ Alpha syntax, subject to change.

D **U** A qualification I owe the reader: I was unable to render the `cassandra.yaml` shipped at the 6.0-alpha2 tag, and therefore cannot assert from a primary source that the `accord` configuration block is present in that specific artefact. What is established is that Accord is absent from that tag's release notes, that its tracking epic is unresolved, and that where it is enabled it is enabled deliberately, per table, rather than by default.

FIGURE 6 — Readiness, and the boundary of what is knowable

Everything left of the dashed line is a fact with a source. Everything right of it is unknown, and no roadmap says otherwise.

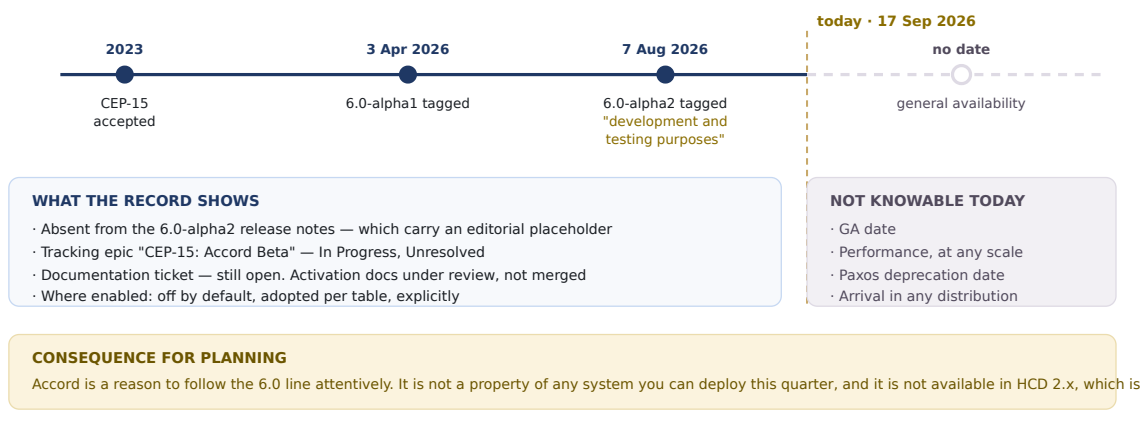


Figure 6. Readiness as evidence rather than as roadmap. **Documented:** the proposal's accepted status (note 23); both alpha tag dates (note 24); the release classification quoted verbatim (note 25); the release-notes contents and their placeholder (note 26); the epic and documentation-ticket states (note 27); the base version of the commercial distribution (note 5). **Reported:** the activation model, from documentation still in review (note 28). The right-hand panel is not a gap in this article's research — it is a gap in the public record, and presenting it as anything else would be the failure mode section 4 exists to name.

3.5 What Accord changes in the five-operation model — and what it does not

Here the analysis becomes unusually clean, because the proposal settles it by its own terms. Among the explicit *non-goals* of CEP-15 are SASI, SAI and local secondary indexes.²³ Storage-attached indexing, which is step four of our five, falls outside the proposal's scope by declaration rather than by inference.

Step	Belongs to	Modified by Accord?
1 — Read the whole document	Storage model	No
2 — Modify in memory	Storage model	No
3 — Rewrite document and derived columns	Storage model	No
4 — Maintain storage-attached indexes	Storage model	No — excluded by CEP-15's stated non-goals
5 — Guard with consensus	Consensus mechanism	Yes, and this step alone

Accord alters how a write is *arbitrated*. It does not alter how a document is *decomposed and rewritten*. The thesis of this article is therefore not weakened by the arrival of a successor protocol; it is corroborated by it. A change of consensus mechanism, a genuine advance in distributed systems, formally specified and years in construction, leaves untouched the four steps that derive from physical layout. That is exactly what one should expect if the layout decision is the real predictor, and exactly what one should not expect if the five-step cost were an artefact of the consensus round. The successor protocol is, in this precise sense, a controlled experiment on the article's own hypothesis.

FIGURE 7 — What a change of consensus protocol does and does not touch

The fifth operation has a designated successor; the first four do not

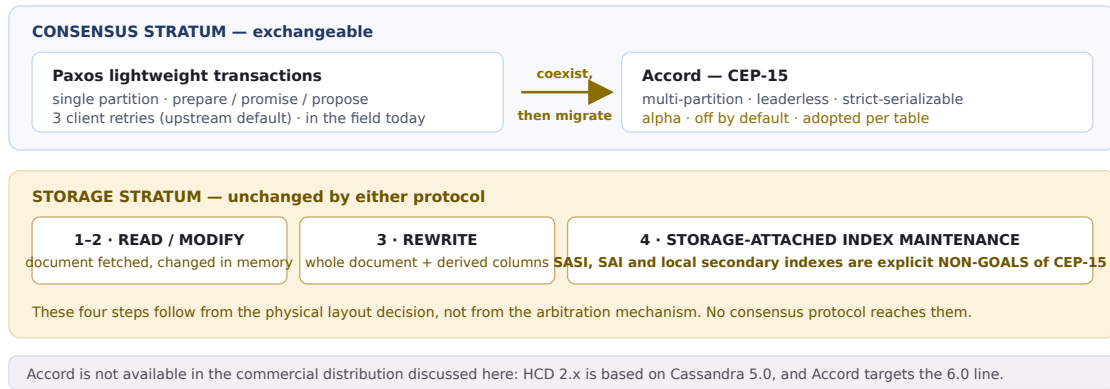


Figure 7. The stratification. **Documented:** Accord's leaderless, strict-serializable, multi-partition character and its explicit non-goals including SAI (note 23); Paxos coexistence and eventual deprecation without timetable (note 23); the retry default (note 1); the base version of the distribution (note 5). **Reported:** the per-table activation model, drawn from documentation still under review (note 28). **Measured, not documented:** the placement of steps 1–3 in the storage stratum, observed by query trace on one build (appendix, measurement M2) and inheriting the **M** marking of Part I, section 4.

D One consequence for readers evaluating the commercial distribution: no mention of Accord appears in the HCD documentation consulted, and HCD 2.0 is based on Cassandra 5.0.⁵ Accord is therefore not available there today. Treat it as a reason to follow the 6.0 line attentively, not as a property of any system you can deploy to production this quarter.

"It would be nice if all of the data which sociologists require could be enumerated because then we could run them through IBM machines and draw charts as the economists do. However, not everything that can be counted counts, and not everything that counts can be counted."

William Bruce Cameron, *Informal Sociology: A Casual Introduction to Sociological Thinking*, Random House, 1963, p. 13.

Routinely attributed to Einstein, who did not write it. An article that grades its own sources should be careful to get right the epigraph it puts above the section on measurement.

4. If you benchmark this yourself

None of what follows is mine, and the authors state it better than I can. The two failures that ruin most published comparisons of these systems are *coordinated omission* — driving load as fast as the system replies, which conceals precisely the stalls under investigation — and reporting an average where a distribution is required. Gil Tene named the first and wrote `wrk2` and HdrHistogram to correct it.³⁰ The remaining pitfalls are, *inter alia*, the documented weaknesses of the Yahoo! Cloud Serving Benchmark on which most published NoSQL comparisons rest.³¹

Four conditions decide whether anyone can interpret your numbers, and all four are disclosure rather than technique: hold durability equivalent between the systems and record the mapping; state the working set against available memory and run both an in-cache and a disk-bound regime; declare the topology, including any coordinating tier, since a stateless API tier is a provisioning decision; and run long enough to cross the background maintenance cycles rather than merely long enough to sustain the rate. The harness accompanying this article applies all of them to itself, and declares in its output record the one it failed to meet.

U That sentence was true of one harness and is not true of the campaigns as a whole, which is worth saying here rather than leaving a reader to find it. Measured against the six requirements above, this study satisfies about half. Load was driven at a fixed offered rate for the freshness probe alone; every mutation, read and comparison measurement ran closed-loop, one operation after the reply to the last — the very failure the first requirement names. One campaign out of seven reached a disk-bound regime and crossed a compaction; the rest are cache figures. Distributions rather than averages, a declared durability mapping and a declared topology are met throughout.

A study that holds others to six rules and meets three of them has less authority than its apparatus suggests, and the right response is to say so in the section that states the rules. It is also the reason the disclosure below promises no benchmark figures in the body: the numbers live in the appendix, each with its conditions, because that is the only place they can honestly stand.³⁸

D For a peer-reviewed comparison of these two database families, the reference is Haughian, Osman and Knottenbelt, *Benchmarking Replication in Cassandra and MongoDB NoSQL Datastores*, DEXA 2016.³² **U** Read it for its stated limitations rather than its results — and note that I withdraw the specific characterisation of those limitations that appeared in an earlier draft of this article, having been unable to obtain the full text and being unwilling to paraphrase a methodology section from an abstract.

5. What to do this week

Measure your write-to-searchable interval. Fifteen minutes.

Run the probe from section 1 against whatever you already operate, once idle and once under write load, and *record the attempt count rather than the elapsed time* — a document found on the first query has no measurable lag, whatever the stopwatch says, and a rising attempt count is the only unambiguous signal that an index is behind. You will obtain a number you almost certainly do not have, and it is the number that decides whether index lag is a constraint you must design around. If you are building anything with retrieval in it, this is the highest-value quarter-hour in either part of this diptych.

And if you deploy across datacentres, measure it there.

Every freshness figure in this part comes from replicas that share a host and answer in under a millisecond. The asymmetry between a quorum write and a single-replica vector read is real in the protocol and was bounded below the measurement floor on that ring. Tens of milliseconds between replicas is exactly the condition this bench could not reproduce, and exactly where the question is open.

6. Conclusion to Part II, and to the diptych

Two kinds of index, and only one is contested. Ordinary secondary indexes are maintained synchronously by both engines: writing a document and querying it through one returned it on the first attempt in every cycle, on both sides. There is parity there and nothing for either party to claim. The asymmetry this part describes is confined to the *search* index — synchronous on one engine, maintained asynchronously through change streams on the other.

The asymmetry is architectural; the magnitude is a deployment property. The asynchronous path carries a periodic refresh: sampled across every phase of its cycle, the write-to-searchable lag spreads from 89 ms to 1 170, mean 646. The synchronous path has no such interval — its index sits on the write path, and no setting would give it one. The first figure this part published, a tight 1 015 ms, was an artefact of a probe that had synchronised itself to the cycle; it overstated the lag by about 1.6 times and is withdrawn.

Whether that costs anything depends entirely on the caller. At zero delay the asynchronous path misses every document and the synchronous one misses none; at half a second, one in two; from a second onward, none at all. The transition tracks the refresh interval of the deployment, not the identity of the product.

Which obliges this part to withdraw its own opening example. Every earlier revision motivated the argument with a conversational assistant. A turn driven by a language model takes seconds; the window closes before the read arrives, and the failure never occurs. What survives is narrower and sharper: where a write and a search are composed inside the refresh window, with no human and no model interposed, the difference is between a right answer and a wrong one, at a rate approaching certainty. Which real workloads sit there is a question this study did not ask.

6.1 The eight axes, and what decides between them

Axis measured	Which engine	Margin, and its reservation	Direction, by separation of support	Beyond the measured envelope?
Mutating one field	MongoDB	3–13× in latency; a memtable-regime figure. The slope ratio is 44× on the pass fitted and 116× on the pass ignored, unbounded there by Fieller	certain — disjoint in 19 of 20 cells	Holds — the cost follows from decomposition itself; more nodes distribute it, none removes it. <i>Unmeasured:</i> under sustained concurrent writes, on either side
Fetching one document by identifier	MongoDB	≈ 20×; allocation between tier and engine unmeasured	certain — disjoint	Undetermined — much of the gap is the interface's HTTP hop, never measured on the read path; it does not depend on volume
Filtering on a declared or wildcard-indexed field	MongoDB	≈ 24×	certain — disjoint	Undetermined — measured at one million documents, cache-

Axis measured	Which engine	Margin, and its reservation	Direction, by separation of support	Beyond the measured envelope?
				resident on both sides
Filtering on a field nobody declared	<i>not established</i>	≈ 22× at the median, and only against the default configuration; what is established is predictability, not speed	not established — supports overlap, ratio interval contains one	Undetermined — a scan degrades with the collection and an index much less, but a direction not established inside the envelope cannot be claimed to establish itself outside it
Freshness of an ordinary secondary index	neither	index current on both; elapsed time is not	tie on attempts; elapsed time disjoint both ways	Undetermined — one datacentre only. Across sites the LOCAL_ONE asymmetry would become observable, <i>against</i> the decomposing engine
Aggregating by group	MongoDB	a capability gap in the <i>interface</i> , not a ratio between	certain — disjoint, on three observations	Holds, and worsens — an absent verb arrives at the first

Axis measured	Which engine	Margin, and its reservation	Direction, by separation of support	Beyond the measured envelope?
		engines: no server-side pipeline; 134 s against 0.2 s by the only route left. The engine's own doctrine says to pre-compute the aggregate at write time, which is neither route		document, and the client scan grows linearly: 10 000 round trips at 200 000 documents, 500 000 at ten million
Counting exactly	MongoDB	by capability: the interface caps exact counts at a documented threshold, and its estimator answered zero before the first flush	structural — no observations, the operation is refused	Undetermined — the refusal is constant, but the comparator's cost depends on the route: its unindexed scan grows with volume while its indexed count, measured at 12.7 ms, does not
Freshness of the search	Decomposing engine	total inside the comparator's refresh	certain by the miss-rate test; not established	Undetermined — the boundary is the refresh

Axis measured	Which engine	Margin, and its reservation	Direction, by separation of support	Beyond the measured envelope?
index (Part II)		window, void outside; that window measured $\approx$ 1 s here	as a latency comparison	interval of a single-node development edition; dedicated search infrastructure may refresh faster or less often

U One column of that table needs its own paragraph, because it decides what a reader may carry away. The measurements were taken on one shared host, at two hundred thousand to one million documents, by a single sequential client, cache-resident but for one campaign, in one datacentre. **That envelope is the one in which the decomposing engine is weakest and the comparator strongest:** nothing that makes the case for a distributed engine — partitioning at scale, sustained concurrent writes, multi-site replication — was exercised once. Three regimes could change the picture and none was measured. When the dataset outgrows one node's memory, where the comparator's unindexed count degrades and its indexed one does not. When write load outgrows what the deployment absorbs — though the comparator here ran as a three-member replica set with a single primary, and a sharded one would not. And across datacentres, where the consensus round measured here at its minimum would cost tens of milliseconds and the single-replica vector read would become observable, *both against the decomposing engine*.

U One caveat governs the table as a whole and is easy to miss because the rows read as commensurable. They are not. The mutation row comes from a three-replica ring in a cache-resident regime; the read and filter rows from a million-document collection against MongoDB 8.0 in a three-member replica set; the aggregation and counting rows from a two-hundred-thousand-document corpus on the same pairing; the search-freshness row from MongoDB 8.3.11 in a single-member deployment, through a different image. Four configurations, one table. The repository's register lists two further measurements with no comparator — where the mutation cost is charged, and vector freshness under a single-replica read — which are properties of one engine rather than axes of comparison and are not counted here. Each row is defensible on its own; no row is a controlled comparison against the others, and a reader should not compute across them. The score, if a reader wants one: five axes to MongoDB, one to the decomposing engine, one to neither, and one the data do not settle.

When the decomposing engine wins. When a write and a search are composed inside the comparator's refresh window — below that boundary one architecture misses everything and the other misses nothing, and no configuration on either side changes it. That is the one axis it wins outright. When the query pattern is not yet known, the index you forgot to declare is the one you never pay for — an operational advantage whose *speed* the data do not establish, only its predictability. And when the workload is append-dominant with retrieval over it, index maintenance being charged once, at insertion.

When MongoDB and its search engine win. When records are mutated repeatedly — counters, status machines, hot aggregates — where the targeted update writes what changed, maintains only the declared indexes, and pays no consensus round. When reads dominate and latency matters. When the query pattern *is* known and the index has been declared. When anything must be aggregated or counted server-side, which the interface does not do at all — Part I, section 4.5. And when the read arrives after the refresh window, which for anything conversational or human-paced it does.

Neither. Freshness of an ordinary secondary index. Both engines maintain those synchronously.

Two numbers decide the contested axis, and only one is about a database: the refresh interval of the deployment you run, and the delay your application leaves between writing and searching. Where the second is smaller than the first, the advantage is total. Where it is larger, there is none.

6.2 The same result, arranged by workload

The table above is arranged by what was measured. This one is arranged by what a reader has. Each row names a workload profile, the engine the measurements favour for it, and the condition under which that preference holds — because every one of them has a condition, and a row read without it is a row misread.

U Do not count the rows. They are profiles, not points, and they do not tally with the eight axes above: three of MongoDB's axes are folded into two rows here, one of the decomposing engine's axes is split across two, and one row for the decomposing engine rests on an inference rather than a measurement. Read row by row, each is defensible. Summed, they would suggest a parity the measurements do not show — the score, if a reader wants one, is the table above: five axes, one, one to neither, and one the data do not settle. The last column says which measured axis each row rests on, and with what evidentiary status.

If your workload is...	Choose	On what evidence, and under what condition	Rests on
Write then search, composed within the refresh window a step that writes and a step that reads, with no human and no model in between	Decomposing engine	The only unconditional win in the diptych: 100 % of documents missed against 0 % at zero delay, still one in two at half a second. <i>Condition:</i> your application's write-to-search delay must be shorter than your deployment's refresh interval. Both are measurements you must take.	M search freshness, inside the window
Counters, status machines, hot aggregates the same record mutated repeatedly	MongoDB	Three to thirteen times cheaper per mutation, widening with indexed volume, and roughly nine tenths of the disadvantage is the engine rather than the interface. <i>Condition:</i> a memtable-regime figure; the gap changes shape once the read leaves memory.	M mutation
Read-dominated, latency-sensitive fetch by key, filter on known fields	MongoDB	About twenty times cheaper by identifier, about twenty-four on a declared or wildcard index. <i>Condition:</i> the split of that gap between the stateless interface and the engine was not measured on the read path.	M point read <i>and</i> declared-field filter — two axes, one row

If your workload is...	Choose	On what evidence, and under what condition	Rests on
Exploratory, schema still moving fields queried before anyone decided to index them	Decomposing engine	Twenty-two times faster than a comparator that must scan a million documents. <i>Condition:</i> the advantage is against a <i>default</i> . Declare a wildcard index on the other side and it is twenty-four times faster still; what you are buying is the absence of a decision, not speed.	M undeclared-field filter
Append-dominant with retrieval over it written once, read and searched many times	Decomposing engine	Index maintenance is charged once, at insertion, and the search index is current at acknowledgement. <i>Condition:</i> this is an inference from two measured properties, not a separately measured workload, and it carries no number of its own.	U inferred; no axis
Conversational, human-paced or model-paced anything where seconds pass between writing and reading	MongoDB	The freshness advantage vanishes entirely once the read arrives after the refresh window, and the mutation and read advantages remain. <i>Condition:</i> none — this is the case the diptych previously got wrong, and correcting it removed the platform's claim on this profile.	M search freshness, outside the window — the other half of row one

If your workload is...	Choose	On what evidence, and under what condition	Rests on
Analytical — aggregate, group, count anything that needs a sum, a group or an exact cardinality server-side	MongoDB	The interface has no aggregation pipeline and caps exact counts at a documented threshold; the only route pulls the collection to the client, 134 s against 0.2 s. <i>Condition:</i> through the Data API as exposed. An analytical engine attached to the same ring, or the engine's native query language over a table designed for the query, were not the object measured.	M aggregation <i>and</i> counting — two axes, one row
Read your own writes through an ordinary index	neither	Both maintain ordinary secondary indexes synchronously; the index is current on both. Elapsed time differs, but that measures the query rather than the index.	M ordinary-index freshness

U Four caveats govern every row. The rows come from three different deployment configurations and are not commensurable with one another. Everything was measured on one host, one build of each engine, with almost no regime outside the cache. Nothing was measured under concurrency, and production is concurrent. And the profiles named in the first column are descriptions rather than measurements: no real application's write-to-search interval was timed, so a reader recognising their own system in a row is making a judgement the study cannot make for them.

6.3 On what this study is worth

It corrected itself four times in public, and each correction cost it something it had claimed. Index maintenance was said to cover the fields that changed; it covers every indexed field. A per-field rate was derived and published; the experiment had grown fields and bytes together and never licensed it. The per-kilobyte rate that replaced it was presented as regime-independent; forcing the read off the memtable halved it. The freshness argument was motivated by conversational retrieval; a sweep of turn latency showed the window closes first. A fifth correction was arithmetic rather than empirical: a sum presented as a replication was an identity, true by construction. Three of the four were found by the author's own follow-up rather than by an objection, which is the only reason they are in the text rather than in a comment thread.

The standard this study sets and does not meet. Section 4 lists six conditions for a defensible comparison; the campaigns satisfy about three. Load was driven closed-loop almost everywhere — one operation awaiting the reply to the last, which is the first thing that section tells a reader not to do. One campaign of seven left the cache. Distributions rather than averages, a declared durability mapping and a declared topology are met throughout. A study that holds others to six rules and meets three has less authority than its apparatus suggests, and saying so is accuracy rather than modesty.

What none of it transposes to. One host, a shared virtual machine whose storage class could not be established. One build of each engine. No concurrency, no multiple clients, no wide-area topology, and — on every axis but one — no confidence interval. Four of the thirty-nine raw files retain their per-observation series; the other thirty-five discarded theirs at write time, irreversibly. The aggregation axis of campaign 7bis is the exception, and it publishes real bootstrap intervals and a rank test. Five of the instruments were written by the person they were meant to constrain, and the verdicts resting on unmodified supplied code are the sounder ones. The comparator was exercised across two deployments, and the search paths compared were not the same object.

The author is employed by a company that sells both engines, and works on one of them. Every figure above is therefore reported with the conditions that would let a reader dismantle it, every correction runs in the text rather than a footnote, and the axis the employer loses is measured in the same detail as the axis it wins. That is the only arrangement under which such a comparison is worth anyone's time — and the reader who concludes, from the five axes that go the other way, that this platform is the wrong choice for their workload will have used this study exactly as intended.

Disclosure. I work at IBM, on the DataStax side of Software, Data & AI. IBM also sells the comparator: MongoDB Enterprise Advanced with IBM is an IBM offering, generally available and supported by IBM,⁵³ so my employer has no commercial interest in one of these two engines prevailing over the other. What is not neutral is my knowledge: I know one of these systems considerably better than the other, and that asymmetry — of expertise, not of interest — is the one the reader should weight both parts of this diptych by. Two deliberate constraints follow. **First**, no benchmark figures appear in the body of either part, because a number produced on my

hardware with my workload would tell you nothing about yours; the measurement appendix reports one estate's observations, each with its conditions, and none of them licenses a comparison with anything. **Second**, every factual claim is graded for evidentiary status and sourced — including claims favourable to my employer, claims I was unable to verify, and two that measurement showed to be wrong.

Corrections are welcome and will be applied with attribution. The apparatus exists so that an error can be demonstrated in minutes rather than argued about, and it is public: every raw measurement record, every probe, the two unmodified originals against which the patched ones can be diffed, the architecture decision record, and the adversarial audits written against this work are at github.com/davidleconte/shredding-cost-hcd-mongodb.

David Leconte — Customer Success Engineer, IBM France · Software, Data & AI | Horizon

Notes

Note numbers run continuously across both parts of this diptych, so a reference keeps its identity wherever it is cited. This part uses 25 of them, and the gaps below belong to the other part rather than to an omission. The appendix that follows carries, for every note, the full locator, the passage relied upon where one exists, and any caveat of scope or edition.

1. DataStax, *About vector databases*, HCD 2.0.
2. DataStax, *Cassandra drivers overview*, HCD 2.0: "HCD 2.x is based on Cassandra 5.0."
3. MongoDB, *Now Source Available: The Engine Powering MongoDB Search*, 15 January 2026. Source of the quoted passage, of the SSPL licensing and of the public-preview status.
4. MongoDB, *MongoDB Search and Vector Search Now Run Anywhere*: `mongot` as a separate binary running alongside `mongod`.
5. MongoDB, MongoDB.local San Francisco 2026: "mongot runs separately from mongod, MongoDB's core database process."
6. OpenSearch documentation, k-NN methods and engines: NMSLIB deprecated in favour of Faiss and Lucene.
7. OpenSearch documentation, breaking changes: default engine changed from NMSLIB to Faiss (2.18); NMSLIB index creation blocked from 3.0.
8. GitHub, `opensearch-project/opensearch-jvector`, README. Source of the DiskANN characterisation, the PQ/BQ support, the thread-safety and incremental-merge claims, and the quoted Cassandra-compatibility passage.
9. Maven Central, `org.opensearch.plugin:opensearch-jvector-plugin`, version 3.7.0.1. Evidences the release train tracking OpenSearch versions.
10. Apache Software Foundation, CEP-30: integration of Lucene's HNSW for vector search in Apache Cassandra.
11. Apache Cassandra project blog, Storage-Attached Indexes: SAI as a new feature in Apache Cassandra 5.0.

12. Apache Software Foundation, CEP-15: General Purpose Transactions. Status: Accepted. Source of the goals, the non-goals (including SASI, SAI and local secondary indexes), the coexistence statement and the long-term Paxos passage quoted in section 3.2.
13. GitHub, `apache/cassandra`, tags: `cassandra-6.0-alpha1` (3 April 2026, commit 1ee7ed7) and `cassandra-6.0-alpha2` (7 August 2026, commit e874e3a).
14. Apache Cassandra release announcement, 6.0-alpha2, 7 August 2026: "This version is a quarterly alpha release on the 6.0 series. Quarterly alphas are intended only for development and testing purposes."
15. GitHub, `apache/cassandra`, `NEWS.txt` at tag `cassandra-6.0-alpha2`. The 6.0 new-features enumeration, the editorial placeholder, and the residual references to 5.1 in the upgrade section.
16. Apache issue tracker, CASSANDRA-17092, epic "CEP-15: Accord Beta", In Progress / Unresolved at consultation; linked CASSANDRA-18229, "Write docs for CEP-15", In Progress.
17. Apache Cassandra pull request 4572, adding `doc/modules/cassandra/pages/developing/cql/transactions.adoc`. Source of `accord: enabled: true` and `WITH transactional_mode = 'full'`. Under review; merge status not established.
18. Apache issue tracker, CASSANDRA-19759. Source of the `BEGIN TRANSACTION` / `LET` / `COMMIT TRANSACTION` syntax, and of the requirement that the table carry `transactional_mode='full'`.
19. GitHub, `giltene/wrk2`. Coordinated omission, constant-throughput load generation, HdrHistogram.
20. Cooper, Silberstein, Tam, Ramakrishnan and Sears, *Benchmarking cloud serving systems with YCSB*, SoCC 2010, pp. 143–154.
21. Haughian, Osman and Knottenbelt, *Benchmarking Replication in Cassandra and MongoDB NoSQL Datastores*, DEXA 2016, LNCS vol. 9828, pp. 152–166, Springer. Full text not consulted.
22. DataStax, how transactions differ, HCD: HCD does not adhere to ACID transactions with rollback or locking mechanisms; a write operation is atomic at the partition level.
23. Measurement record, single-node campaign: `findings.json`, run 2026-09-17T14:59Z, one node at RF = 1. Source of measurements M1 to M4 and M9, whose

full entries are in Part I — the twelve columns and nine automatic indexes, the statement trace, the cost-against-unchanged-content probe with its read controls and payload sizes, the 8 000-byte indexed-string limit, the consistency levels, and the freshness decomposition. Conditions in the measurement appendix.

24. Measurement record, three-replica campaign: `findings_rf3.json` and its report, 2026-09-17, six-node two-datacentre ring with the working keyspace at three replicas. Source of M5, the Paxos round and its coordinator-side figures, and of M6, the Data API tier's startup configuration showing the consistency properties and their asymmetry.
25. Measurement record, vector freshness: `findings_vector_rf3.json` and `vector_freshness_rf3.py`, 2026-09-17, same three-replica ring. Source of M7 — 120 cycles in two regimes, with the similarity and rank controls that establish the probe was measuring what it claims.
26. Measurement record, read and search axes: `probe_read_search.py`, 18 September 2026, one million documents per arm on one host, MongoDB as a three-member replica set at majority write concern. Source of the ordinary-secondary-index parity reported in section 1.1, and of the finding that MongoDB's search engine could not be deployed on a plain replica set, leaving the search-freshness axis unmeasured. Full entries in Part I, measurements M15 and M16.
27. Measurement record, search freshness: `mongot_freshness.py` and the companion vector-freshness probe, 18 September 2026, one host and one session. MongoDB 8.3.11 through the vendor's packaged local image, which bundles the database with its search process; forty cycles each side. Source of measurement M17.
28. Measurement record, the refresh interval: `mongot_floor.py`, 18 September 2026. Sixty cycles with a random delay of up to 1 200 ms before each write, so that every phase of the indexing cycle is sampled, then polling at 5 ms. Source of measurement M18, and of the correction to M17.
29. Measurement record, the failure rate against turn latency: `turn_latency.py`, 18 September 2026. Nine delays between write and search, thirty cycles at each, both engines on one host. Source of measurement M19.
30. MongoDB, *Supercharge Self-Managed Apps With Search And Vector Search Capabilities*, 30 September 2025. States that the managed service offers a production-ready version of the capabilities that were then entering self-managed public preview. <https://www.mongodb.com/company/blog/product-release-announcements/supercharge-self-managed-apps-search-vector-search-capabilities>

31. MongoDB, *Release Notes — Self-Managed MongoDB Search and Vector Search*. Covers the public preview release 0.40.0 of October 2025 through the general availability release 1.70.1. <https://www.mongodb.com/docs/search/self-managed/current/release-notes/>
32. MongoDB, *MongoDB Search and Vector Search with the Kubernetes Operator — FAQ*. States that 1.70.1 is the minimum `mongot` version for self-managed deployments and that earlier 0.x builds are preview releases not recommended for production. The page reachable at consultation was a Spanish-locale copy of the “upcoming” documentation version, which is weaker provenance than a current English page, and the claim is graded accordingly. <https://www.mongodb.com/es/docs/kubernetes/upcoming/fts-vs/faq/>
33. Image provenance, recorded 18 September 2026 from the running container: `mongot` version 1.75.1 from the image label and from `/opt/mongot/mongot --version`; edition `localDev` from the image label and from `/etc/mongodb-atlas-local/mongot-edition`; `mongod` 8.3.11 Community from the image label; image digest `sha256:e118f5c131c5004e4ccd0554d3f32b048b4af6a33325e80e424eaa1c8986d8f1` from the repository digests; build 2026-09-16T08:14:20Z, vcs-ref 28eadd3b, from the labels.
34. MongoDB, *Search Nodes Now In Public Preview: Performance At Scale With Dedicated Infrastructure*, with its update of 25 June 2024. Source of the statement that Search Nodes provide dedicated infrastructure for Search and Vector Search workloads, separate from the database, and of their general availability on AWS, Google Cloud and Microsoft Azure. <https://www.mongodb.com/company/blog/product-release-announcements/search-nodes-now-public-preview-performance-scale-dedicated-infrastructure>
35. MongoDB, *Configure mongot for MongoDB Vector Search Automated Embedding*. Source of the statement that, with Automated Embedding, the search process generates vector embeddings for text data at index time and for query text at query time, and of its preview status for self-managed deployments. <https://www.mongodb.com/docs/search/self-managed/current/configuration/automated-embedding/>
36. IBM, *MongoDB Enterprise Advanced with IBM*, product page, and IBM Support, *MongoDB Enterprise Advanced with IBM 7.0.x lifecycle* page, which states that the 7.0.x offering is generally available as of 29 April 2025, can be purchased and is fully supported by IBM. <https://www.ibm.com/products/mongodb-enterprise-advanced> and <https://www.ibm.com/support/pages/mongodb-enterprise-advanced-ibm70x>

37. Repository, `data/derived/inference.json` and `docs/STATISTICS.md` : re-analysis of every latency comparison from the surviving summary statistics by separation of support, and Fisher's exact test on the miss counts of measurement M19. The freshness comparison in its de-synchronised form is overlapping; the miss-rate at zero delay is exactly tested at a one-sided probability of about 10^{-17} . The same document records that the LLM-turn premise of section 1.2 is an assumption of the challenge author, unmeasured. <https://github.com/davidleconte/shredding-cost-hcd-mongodb>
38. MongoDB, *Known Limitations for Self-managed mongot*. States that Search and Vector Search share the same engine on the managed service and on self-managed deployments, with consistent query behaviour, scoring and index semantics. <https://www.mongodb.com/docs/search/self-managed/current/limitations/>
39. MongoDB, *MongoDB Search and MongoDB Vector Search on Self-Managed Deployments*. Designates the `mongodb/mongodb-atlas-local` image for evaluation only, as a single node without high availability and not a production deployment target. <https://www.mongodb.com/docs/search/self-managed/current/>

Appendix — sources

Each entry records what the source establishes, the edition or version to which it pertains, and the locator. All were consulted on 17 September 2026 unless otherwise stated. Numbering matches the notes above and is continuous across both parts.

4 — DataStax, *About vector databases*

Establishes: immediate addition of newly inserted documents to the vector graph, attributed to JVector; quantisation support.

"When you insert data, JVector adds those new documents to the graph immediately, so you can efficiently search right away."

Version: HCD 2.0.

<https://docs.datastax.com/en/hyper-converged-database/2.0/get-started/vector-concepts.html>

5 — DataStax, *Cassandra drivers overview*

Establishes: the base engine version of the distribution.

"HCD 2.x is based on Cassandra 5.0."

Version: HCD 2.0.

<https://docs.datastax.com/en/hyper-converged-database/2.0/drivers/cassandra-drivers-overview.html>

14 — MongoDB, *Now Source Available: The Engine Powering MongoDB Search*

Establishes: the asynchronous indexing mechanism; the SSPL licensing; the public-preview status; the date.

"mongot builds and maintains these compute-intensive indexes outside of the transaction commit window by leveraging change streams to asynchronously replicate changes to source data, allowing customers to add search and vector search capabilities without fear of impacting existing transactional workloads."

Date: 15 January 2026.

<https://www.mongodb.com/company/blog/product-release-announcements/now-source-available-the-engine-powering-mongodb-search>

15 — MongoDB, *search and vector search running anywhere*

Establishes: `mongot` as a separate binary running alongside `mongod`.

<https://www.mongodb.com/company/blog/product-release-announcements/mongodb-search-vector-search-now-run-anywhere>

16 — MongoDB.local San Francisco 2026

Establishes, independently: process separation.

"mongot runs separately from mongod, MongoDB's core database process"

<https://www.mongodb.com/company/blog/events/mongodb-local-san-francisco-2026-ship-production-ai-faster>

17 — OpenSearch, k-NN methods and engines

Establishes: the three core engines and the deprecation of NMSLIB.

"While still supported, it has been deprecated in favor of the Faiss and Lucene engines."

Version caveat: documentation published at the unpinned "latest" path; pinned by entry 18.

<https://docs.opensearch.org/latest/mappings/supported-field-types/knn-methods-engines/>

18 — OpenSearch, breaking changes

Establishes: the default-engine change and the version at which NMSLIB index creation is blocked.

"The default k-NN engine changed from NMSLIB to Faiss"

Version: change at 2.18; blocking from 3.0.

<https://docs.opensearch.org/latest/breaking-changes/>

19 — `opensearch-project/opensearch-jvector`, README

Establishes: DiskANN family, PQ and BQ support, thread safety, incremental merges, Apache-2.0 licensing, Cassandra compatibility.

"Compatibility - JVector is compatible with Cassandra. Which allows to more easily transfer vector encoded data from Cassandra to OpenSearch and vice versa."

<https://github.com/opensearch-project/opensearch-jvector>

20 — Maven Central, OpenSearch JVector plugin

Establishes: the release train tracking OpenSearch versions.

Version: 3.7.0.1.

<https://central.sonatype.com/artifact/org.opensearch.plugin/opensearch-jvector-plugin>

21 — Apache Software Foundation, CEP-30

Establishes: upstream Cassandra vector search built on Lucene HNSW, not JVector.

"Integrate Lucene's HNSW: The implementation will leverage Lucene's Hierarchical Navigable Small World (HNSW) library"

Version: Apache Cassandra 5.0.

<https://cwiki.apache.org/confluence/pages/viewpage.action?pageId=255069753>

CEP-30 page (Apache Confluence):

<https://cwiki.apache.org/confluence/download/attachments/188744725/Accord.pdf>

22 — Apache Cassandra project blog, Storage-Attached Indexes

Establishes: SAI as a 5.0 feature.

"Storage Attached Indexes, or SAI, are a new feature in Apache Cassandra 5.0"

https://cassandra.apache.org/_/blog/Apache-Cassandra-5.0-Features-Storage-Attached-Indexes.html

23 — Apache Software Foundation, CEP-15: General Purpose Transactions

Establishes: the goals (strict-serializable isolation, one wide-area round trip, live migration from Paxos); the non-goals (SASI, SAI and local secondary indexes); the Paxos coexistence and long-term deprecation language; the reorder buffer and fast-path electorate techniques; the EPaxos lineage.

"Initially, supporting the Paxos-based LWT and Accord side by side is also an example of such modularity and optionality."

Status: Accepted. Tracking JIRA: CASSANDRA-17092.

<https://cwiki.apache.org/confluence/display/CASSANDRA/CEP-15:+General+Purpose+Transactions>

24 — GitHub, `apache/cassandra`, tags

Establishes: the alpha tag dates.

Versions: `cassandra-6.0-alpha1`, 3 April 2026, commit 1ee7ed7; `cassandra-6.0-alpha2`, 7 August 2026, commit e874e3a.

<https://github.com/apache/cassandra/tags>

25 — Apache Cassandra, release announcement 6.0-alpha2

Establishes: the quarterly-alpha classification and its intended use.

"Quarterly alphas are intended only for development and testing purposes."

Date: 7 August 2026. Scope caveat: consulted through a list mirror rather than the ASF archive directly.

<http://www.mail-archive.com/user@cassandra.apache.org/msg64289.html>

26 — GitHub, `apache/cassandra`, `NEWS.txt` at tag `cassandra-6.0-alpha2`

Establishes: the enumerated 6.0 new features, from which Accord is absent; the editorial placeholder that prevents this absence from being probative; the residual 5.1 references evidencing renumbering.

"[The following is a placeholder, to be revised asap]"

<https://github.com/apache/cassandra/blob/cassandra-6.0-alpha2/NEWS.txt>

27 — Apache issue tracker, CASSANDRA-17092 and CASSANDRA-18229

Establishes: the CEP-15 tracking epic titled "CEP-15: Accord Beta", In Progress and Unresolved at consultation; the documentation ticket likewise In Progress.

<https://issues.apache.org/jira/browse/CASSANDRA-17092>

Linked documentation ticket: <https://issues.apache.org/jira/browse/CASSANDRA-18229>

28 — Apache Cassandra pull request 4572

Establishes: the Accord activation model — global `accord: enabled: true` and per-table `transactional_mode`.

"When a table has transactional_mode='full', your existing CQL statements are automatically executed through Accord."

Status caveat: documentation under review; merge status not established. Consulted through the project's pull-request list mirror.

<https://github.com/apache/cassandra/pull/4572>

29 — Apache issue tracker, CASSANDRA-19759

Establishes: the explicit transaction syntax and the requirement that the table be marked transactional.

"the table was not marked as an Accord table; aka missing ... WITH transactional_mode='full'"

Version caveat: alpha syntax, subject to change before general availability.

<https://issues.apache.org/jira/browse/CASSANDRA-19759>

30 — GitHub, `giltene/wrk2`

Establishes: the attribution of coordinated omission and the corrective design of the tool.

"The model I chose for avoiding Coordinated Omission in wrk2 combines the use of constant throughput load generation with latency measurement that takes the intended constant throughput into account."

<https://github.com/giltene/wrk2>

31 — Cooper, Silberstein, Tam, Ramakrishnan, Sears

Benchmarking cloud serving systems with YCSB. 1st ACM Symposium on Cloud Computing (SoCC), Indianapolis, 2010, pp. 143–154. DOI 10.1145/1807128.1807152.

Establishes: the origin and documented limitations of YCSB.

<https://doi.org/10.1145/1807128.1807152>

Open copy of the paper: <https://courses.cs.duke.edu/fall13/compsci590.4/838-CloudPapers/ycsb.pdf>

32 — Haughian, Osman, Knottenbelt

Benchmarking Replication in Cassandra and MongoDB NoSQL Datastores. DEXA 2016, LNCS vol. 9828, pp. 152–166, Springer, Cham. DOI 10.1007/978-3-319-44406-2_12.

Establishes: the bibliographic reference only. Scope caveat: full text not consulted; no characterisation of the paper's methodology or stated limitations is made in this article.

https://doi.org/10.1007/978-3-319-44406-2_12

33 — DataStax, how transactions differ

Establishes: the absence of ACID transactions with rollback or locking; partition-level write atomicity; row-level isolation.

Version: HCD.

<https://docs.datastax.com/en/hyper-converged-database/1.1/architecture/database-internals/transactions-differ.html>

45 — MongoDB, product announcement, self-managed search preview

Establishes: at the self-managed public preview of 30 September 2025, the managed service already offered a production-ready version of the same capabilities. Consulted 18 September 2026.

<https://www.mongodb.com/company/blog/product-release-announcements/supercharge-self-managed-apps-search-vector-search-capabilities>

46 — MongoDB, self-managed search release notes

Establishes: the self-managed capability moved from public preview (0.40.0, October 2025) to general availability (1.70.1). Consulted 18 September 2026.

<https://www.mongodb.com/docs/search/self-managed/current/release-notes/>

47 — MongoDB, known limitations for self-managed mongot

Establishes: the search engine is shared between the managed service and self-managed deployments, with consistent query behaviour, scoring and index semantics. Consulted 18 September 2026.

<https://www.mongodb.com/docs/search/self-managed/current/limitations/>

48 — MongoDB, self-managed search overview

Establishes: the packaged local image is for evaluation only, single-node, without high availability, and is not a production deployment target. Consulted 18 September 2026.

<https://www.mongodb.com/docs/search/self-managed/current/>

49 — MongoDB, Kubernetes operator FAQ for Search and Vector Search

Establishes: 1.70.1 as the minimum self-managed `mongot` version, 0.x builds being preview releases not recommended for production. Reached only as a Spanish-locale copy of the “upcoming” documentation version; graded R rather than D on that account. Consulted 18 September 2026.

<https://www.mongodb.com/es/docs/kubernetes/upcoming/fts-vs/faq/>

51 — MongoDB, Search Nodes announcement

Establishes: Search Nodes as dedicated infrastructure for Search and Vector Search on the managed service, separate from the database, generally available on AWS, Google Cloud and Azure per the post’s June 2024 update. Consulted 18 September 2026.

<https://www.mongodb.com/company/blog/product-release-announcements/search-nodes-now-public-preview-performance-scale-dedicated-infrastructure>

52 — MongoDB, Automated Embedding configuration for self-managed mongot

Establishes: with Automated Embedding, the search process generates embeddings at index time and at query time in place of the application; the feature is in preview for self-managed deployments and may change. Consulted 18 September 2026.

<https://www.mongodb.com/docs/search/self-managed/current/configuration/automated-embedding/>

53 — IBM, MongoDB Enterprise Advanced with IBM

Establishes: that IBM sells and supports the comparator under its own name, the 7.0.x offering being generally available as of 29 April 2025 per IBM’s lifecycle page. Bears on the disclosure only. Consulted 18 September 2026.

<https://www.ibm.com/products/mongodb-enterprise-advanced> ·
<https://www.ibm.com/support/pages/mongodb-enterprise-advanced-ibm70x>

Appendix B — measurements

Each entry records what was observed, on which build, under which conditions, and where the raw record lives. A measurement establishes what one build did on one day; it is not a vendor's documentation and does not license comparison with any other product. Identifiers M1 to M8 are the same identifiers used in the architecture decision record that accompanies the raw files, and they are continuous across both parts of this diptych.

Common conditions

IBM DataStax HCD 2.0.6, release string `5.0.7.0-ea50e91ba01f` read from the running node (`nodetool version`, `system.local`); Data API `stargateio/data-api` v1.0.33; astrapy 2.3.1. One node, one datacentre, RF 1 (`NetworkTopologyStrategy {dc1: 1}`), 16 vnodes. Virtual machine, 80 vCPU Intel Xeon Gold 6148 @ 2.40 GHz, 220 GiB RAM; the database container limited to 3 GiB with a 2 GiB heap; storage class of the backing media not determinable from the guest. `commitlog_sync periodic`, 10 000 ms. Working set memtable-resident throughout: no SSTable was flushed and no compaction crossed. Load driven at a fixed offered rate of 50 writes/s for 120 s, open loop, schedule slip under 0.06 s. Client on the same host over loopback. Harness `verify_storage_claims.py` 1.0 with two mechanical fixes (client environment set to HCD; probe-3 ballast excluded from indexing because the Data API refuses any indexed string above 8 000 bytes). 17 September 2026.

Full record: `findings.json`.

M2 — The write path of a single-field update

Establishes: with query tracing at probability 1.0 on the node, each of ten `updateOne` operations carrying one `$set` produced, in order, a `SELECT` of the whole document by key and then one `UPDATE` rewriting every derived column and `doc_json`, conditioned on `tx_id`. Each `deleteOne` produced a `SELECT` of `key, tx_id` then `DELETE ... IF tx_id = ?`. Each `insertOne` produced one `INSERT ... IF NOT EXISTS`. Tracing was reset to 0.0 afterwards. Ten of ten updates showed the pair.

```
SELECT key, tx_id, doc_json FROM ... WHERE key = ? LIMIT 1 — then — UPDATE ...
SET tx_id = now(), exist_keys = ?, array_size = ?, array_contains = ?,
query_bool_values = ?, query_dbl_values = ?, query_text_values = ?,
query_null_values = ?, query_timestamp_values = ?, query_lexical_value = ?, doc_json
= ? WHERE key = ? IF tx_id = ?
```

Conditions for M5 to M8 — the three-replica ring and the nested-document read

M7, M8 and M9 were taken on a six-node, two-datacentre ring, the working keyspace at three replicas in one datacentre, one replica per rack; nodes larger than the single-node host used for M1 to M6 (8 GiB with a 4 GiB heap against 3 GiB with a 2 GiB heap), which means end-to-end latencies across the two campaigns mix a replication effect with a hardware difference and must not be compared directly. Coordinator-side figures and protocol traces are structural and do not carry that confound. M8 was read on the single node. Every regime memtable-resident; tracing raised to 1.0 and restored to 0.0 on every node touched; keyspaces dropped and snapshots cleared afterwards. 17 September 2026.

M6 — The Data API's consistency levels are configuration, and they are asymmetric

Establishes, from the tier's own startup configuration: writes, ordinary reads and schema changes default to `LOCAL_QUORUM`; serial consistency to `LOCAL_SERIAL`; the lightweight-transaction retry count to three; and *vector search* to `LOCAL_ONE`. All are settable properties rather than fixed behaviour. The asymmetry between a quorum write and a single-replica vector read is the subject of M7.

M7 — Vector-search freshness under that asymmetry

Establishes: on the three-replica ring, with a sixteen-dimension cosine collection, the just-written vector was the top match on the first `LOCAL_ONE` query in 120 of 120 cycles — sixty idle, sixty under forty vector writes per second — and in no cycle did the quorum key read find the document while the vector search missed it. Insert-to-visible median 74 ms, HTTP-bound. Validity controls: the marker returned at cosine similarity 1.0 in rank one, background vectors at 0.83 or below, and an unrelated query returned a different top match.

Verdict: **not detectable**. This neither confirms nor refutes the searchability claim of section 5. It bounds the window below the measurement floor on a ring whose replicas share one host and answer in under a millisecond, and it names the wide-area topology this bench cannot reproduce as the place where the question is actually open.

M17 — Search freshness, both engines, one session

Establishes: writing a document and then querying for it through the search index, forty cycles each side on one host. The decomposing engine returned it on the first query in every cycle, at a median of 45 ms write-to-searchable, which bounds its index lag below one HTTP round trip without establishing that the lag is nil; its dispersion is wider than the comparator's, at a ninety-fifth percentile of 85 ms and a maximum of 574. MongoDB's `$search` required a median of thirty-four polling attempts and 1 015 ms; no cycle failed to become visible within the timeout. That figure was later shown to be an artefact of the probe and is superseded by measurement M18. The same documents were retrievable by identifier on MongoDB in 1.3 ms throughout, which isolates the lag to the search index rather than to the write.

Method: MongoDB 8.3.11 Community through the vendor's packaged local image, bundling `mongot` 1.75.1 in its `localDev` edition (image digest `sha256:e118f5c131c5004e4ccd0554d3f32b048b4af6a33325e80e424eaa1c8986d8f1`, built 16 September 2026) — a generally-available-line version of the engine, in an edition the image labels as for local development; a search index was created and confirmed ready, and a query validated against it, before any measurement was taken.

Not established: the magnitude of the asynchronous lag on any other deployment — the image used is the vendor's development and testing build, and a quantity that depends on the deployment cannot be vouched for by one of them; behaviour under load, the figure being an idle one, which would widen the lag rather than narrow it; the effect of replica count, the packaged image running a single member so that a majority write concern is satisfied trivially there; the index lag of the decomposing engine, which is bounded below the measurement floor rather than measured; and equivalence of the search paths, a vector index having been exercised on one side and a text index on the other, so that the comparison of ingestion architectures is an inference rather than an observation. What the two share, and what is being compared, is that one maintains its index on the write path and the other replicates to it asynchronously.

Raw record: `findings_mongot_freshness.json`, `findings_hcd_vec_freshness.json`. — published in the repository, github.com/davidleconte/shredding-cost-hcd-mongodb

What the measurements do not establish

Any disk-bound behaviour: every regime was memtable-resident and no compaction was crossed. The separation of a per-field from a per-byte cost in the indexing term, which Part I now resolves in favour of bytes. The internal decomposition of the Data API tier's own cost, which Part I separates from storage but does not break down into parsing, shredding and serialisation. The vector-search window under real wide-area latency, M9 having been taken on co-located replicas, and with it the separation of replication lag from index-build lag — M9 saw no misses, so the two could not be told apart. A client-side figure for the Paxos round. Anything about another version of HCD or the Data API, about Apache Cassandra 5.0 or 6.0, or about MongoDB.

M18 — The asynchronous lag is a refresh interval, and the first figure overstated it

Establishes: inserting a random delay of up to 1 200 ms before each write, so that every phase of the indexing cycle is sampled rather than one, gives a lag spread from 89 ms to 1 170 ms with a mean of 646 and a standard deviation of 291 across sixty cycles. The signature is a periodic refresh of roughly a second, not a fixed pipeline delay.

What it corrects. The probe behind M17 wrote, polled until the document appeared, and wrote again — a loop that synchronises itself to the cycle, so that every write landed just after a commit and waited a full period. Its tight 1 015 ms was the worst phase of the cycle, and overstated the lag by about 1.6 times. The error was the measurer's, found by his own follow-up rather than by an objection.

Consequence for the comparison: the mean gap against the synchronous index is about fourteen times rather than twenty-two, and the best-case gap about two — 89 ms against a 45 ms floor. The direction is unchanged and the margin is narrower.

Not established: the interval on any other deployment or edition. The engine measured is `mongot` 1.75.1 in its `localDev` edition; no configuration knob for the interval was found in the packaged image, so it is that edition's internal default, and whether the managed service or a self-managed production edition commits on the same period is not known.

Raw record: `findings_mongot_floor.json`. — published in the repository, github.com/davidleconte/shredding-cost-hcd-mongodb

M19 — What the window costs, against how soon the read arrives

Establishes: sweeping the delay between the write and the search, thirty cycles at each of nine delays, both engines on one host. The decomposing engine missed no document at any delay, including zero. The asynchronous path missed every document at zero delay, 93 % at 100 ms, 83 % at 250, 50 % at 500, 33 % at 750, and none from 1 000 ms onward. The transition sits at the refresh interval measured in M18, and is located only to within a quarter of a second, the flanking points being 750 ms and 1 000 ms. Thirty cycles per point gives a confidence interval of roughly eighteen points around the middle values. The zero-delay row compares a native driver against an HTTP interface and therefore not the same instant; corrected for the forty-five millisecond difference it would read about 97 %.

What it withdraws. Earlier revisions of this part motivated the freshness argument with conversational retrieval — an assistant writing a fact and searching for it on the following turn. A turn driven by a language model takes seconds, so the window closes before the read arrives and the failure never occurs. That motivation is withdrawn. The advantage holds for writes composed with searches inside the refresh window, with no human and no model interposed. Which real workloads sit there is a question this study did not ask: no application's write-to-search interval was measured, and naming candidate workloads — tool chains, ingestion loops — would repeat, one line later, the error this measurement just corrected.

Not established: the position of the transition on any other deployment, since it follows the refresh interval of this one; and whether any particular production workload sits below or above it, which is a property of the application rather than of either engine.

Raw record: `findings_turn_mongodb.json`. — published in the repository, github.com/davidleconte/shredding-cost-hcd-mongodb

Limitations

Stated here rather than buried, because work that grades its claims owes the reader an account of where the grading came out badly. These limitations apply to the diptych as a whole.

Four substantive claims could not be verified against sources of acceptable rank, and three of the four concern the decomposing engine rather than its comparator: the enumeration of generic column names, the read-modify-write cycle, the consistency levels at which Data API writes execute, and the current general-availability status of MongoDB search per edition, since resolved from the vendor's pages (section 1.3). The first three were subsequently measured and hold on the builds named; they carry the **M** marker, which is weaker than **D**: an observation of a build is not a vendor's commitment. The fourth has since been resolved from the vendor's own pages, and the paragraph above is corrected accordingly.

That the unverified residue falls almost entirely on one side is itself a finding, and it cuts against the side I work for. MongoDB's mutation semantics, its index policy and its asynchronous search path are documented in version-pinned reference material and were confirmed verbatim. The equivalent statements about the Data API's physical representation are not published anywhere I could reach. A reader may reasonably conclude that the system I know better is the one whose mechanism is harder to verify, and may weight the comparison accordingly. An earlier revision added that the asymmetry ran further still, nine measurements having been taken on one side and none on the other. That is no longer so: the comparator has since been measured on eight axes, and the results are in the tables of section 6. The documentary asymmetry above remains; the measurement asymmetry does not.

The write-cost measurement that Part I reports has since been resolved in one direction. A later campaign separated the two terms that the original run had grown together and found the cost carried by indexed *bytes* rather than by field count, at roughly 0.77 ms per indexed kilobyte. Readers who met an earlier revision of either part will have seen a per-field rate; it was withdrawn, and the correct unit is in Part I. Later campaigns separated where that cost is paid — a minority in the stateless tier, the majority in the stateful engine, the exact share unsettled between a tenth and a third — then found the rate's boundary: it is a memtable-regime quantity that collapses behind a large fixed read cost once the cycle's internal read leaves memory. They also measured the comparator at last: on the mutation axis the decomposing engine costs between roughly three and thirteen times a capability-matched MongoDB, the gap widening with indexed volume — a range of latencies, the slope ratios that circulate from this work being unsupported by anything in it. All of that is in Part I.

This study does not meet the standard it sets in section 4. Of the six requirements listed there, roughly three are met across the campaigns: distributions are reported, durability is mapped and declared, topology is declared. Load was driven closed-loop almost everywhere, one operation awaiting the reply to the last, which is the first thing that section tells a reader not to do; and only one campaign of seven left the cache. The directions reported in both parts survive that; the magnitudes are worth less than their decimal places suggest, and a reader should treat every figure here as an observation of one build on one machine rather than as a property of a product.

The axis on which this part rests was measured late, and on one deployment. An earlier revision recorded that it had never been measured at all, MongoDB's search process appearing undeployable outside its managed service; the vendor's own packaged local image made it available, and section 1.1 reports the result. The figure that matters — a periodic refresh averaging two thirds of a second, against none — rests on a single-member deployment, an idle system, a development edition of the engine (`mongot` 1.75.1, `localDev`), and two different search paths, a vector index against a text one. The architecture being compared is the same on both; the objects being searched are not. Load would widen the lag rather than narrow it, which means the reservation runs in the direction favourable to the conclusion, and a reader should discount accordingly rather than the reverse.

Every figure in this part was taken in a memtable-resident regime, and Part I has since shown that this is not a detail: the write-cost rate it reports collapses once the read leaves memory, so no quantity in either part should be read as a disk-regime figure. No SSTable was flushed and no compaction was crossed for any measurement reported here, so these are in-memory latencies. A disk-bound run at three replicas has since been executed for the write-cost measurements of Part I, where the indexed-content coefficient did not collapse; the freshness and consensus measurements of this part were not repeated there. A page-cache confound also survives in that later run, the operating system cache never having been dropped.

Two product pages could not be rendered during verification — the HCD 2.0 Data API limits page and the HCD 2.0 data-model page — so several product figures rest on the equivalent commercial documentation for the same API surface rather than on the exact edition under discussion. These substitutions are marked in the appendix entry concerned.

Everything concerning Cassandra 6.0 describes software in alpha, explicitly designated by its publishers as unsuitable for anything beyond development and testing. Its configuration surface, its syntax and its feature scope may all change before general availability.

Two sources were consulted through list mirrors rather than the Apache archive directly, and one through the project's pull-request traffic rather than published documentation. Where that provenance bears on the weight of a claim, the claim carries the **R** marker rather than **D** — with one exception I would rather name than let a reader find. The alpha classification quoted in Part II is marked **D** although the only reachable copy of that announcement was a list mirror. The quotation is verbatim and the tag dates corroborate it from the project repository itself, but by the rule stated in this very paragraph it ought to carry **R**. Consistency is a property of a system of markers, not of any one of them, and this is where mine failed.

The interactive control in section 1 measures nothing. Its six index-lag values are plausible orders of magnitude chosen to span the space; they come from the author, not from any system, and the zero-lag setting corresponds to a behaviour this work could establish only below a detection floor of roughly one network round trip. It is a demonstrator of a mechanism, and the rule about diagrams carrying unearned authority applies to it with more force than to any static figure.